

Отчет по лабораторной работе №1
по курсу «Алгоритмы и структуры
данных»

Тема: Сортировка вставками, выбором,
пузырьковая

Выполнил:

Афанасьев Д.М.

К3140

Проверил:

##

Санкт-Петербург

2024 г.

Содержание

Содержание.....	2
Задание 1.....	3
Задание 2.....	5
Задание 3.....	7
Задание 4.....	9
Задание 5.....	11
Задание 6.....	14

Задание 1

Используя код процедуры Insertion-sort, напишите программу и проверьте сортировку массива $A = \{31, 41, 59, 26, 41, 58\}$.

```
def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

def insertionSort() -> None:
    file = open("input.txt").readlines()[1]
    #N = int(open("input.txt").readlines()[0])
    array = list(map(int, file.split()))
    length = len(array)
    for index in range(1, length):
        tmp = index
        while array[tmp] < array[tmp - 1]:
            array[tmp], array[tmp - 1] = array[tmp - 1], array[tmp]
            tmp -= 1
        if tmp == 0:
```

```

        break
    open("output.txt", "w").write(normVid(array))

```

- NormVid преобразует массив в строку, которая должна получиться на выходе
- сама сортировка происходит по образцу : Сравниваем 2 элемента, затем либо меняем их местами, либо нет

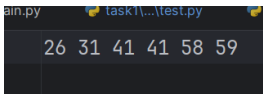
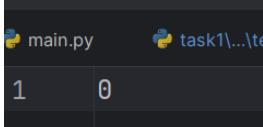
Тесты

в качестве максимальных данных, допустимых в условии задачи создавался массив следующим образом:

```

array = [random.randint(-10 ** 9, 10 ** 9) for i
in range(N)]

```

ввод	Время	Память	Вывод
Пример из задачи	0.000581502914 4287109	22.69 МВ	
Минимальные данные	0.00098109	22.68 МБ	
Максимальные данные	0.021698236465 4541	22.84 МВ	...

-

Задание 2

Измените процедуру Insertion-sort для сортировки таким образом, чтобы в выходном файле отображалось в первой строке n чисел, которые обозначают новый индекс элемента массива после обработки.

```
def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

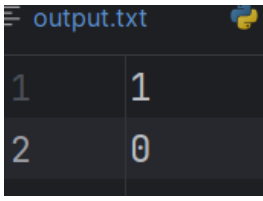
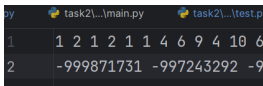
def insertionSort() -> None:
    file = open("input.txt").readlines()[1]
    array = list(map(int, file.split()))
    length = len(array)
    b = []
    for index in range(length):
        tmp = index
        while array[tmp] < array[tmp - 1]:
            array[tmp], array[tmp - 1] = array[tmp - 1], array[tmp]
            tmp -= 1
        if tmp == 0:
            break
        b.append(tmp + 1)
```

```

string_output = normVid(b) + "\n" +
normVid(array)
open("output.txt", "w").write(string_output)

```

- Аналогично с предыдущей задачей, только мы сохраняем индексы при swape элементов

ввод	Время	Память	Вывод
Пример из задачи	0.001006603240	15.29 МБ	
Минимальные данные	0.001045465469 3	15.21 МБ	
Максимальные данные	0.263513565063 47656	15.43 МБ	

-
-

Задание 3

Перепишите процедуру Insertion-sort для сортировки в невозрастающем порядке вместо неубывающего с использованием процедуры Swap.

Формат входного и выходного файла и ограничения - как в задаче 1.

```
def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

def insertionSort() -> None:
    file = open("input.txt").readlines()[1]
    #N = int(open("input.txt").readlines()[0])
    array = list(map(int, file.split()))
    length = len(array)
    for index in range(1, length):
        tmp = index
        while array[tmp] > array[tmp - 1]:
            array[tmp], array[tmp - 1] = array[tmp - 1], array[tmp]
```

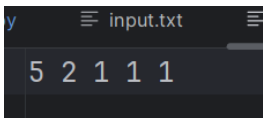
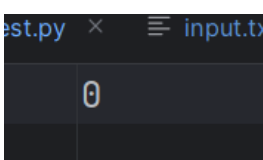
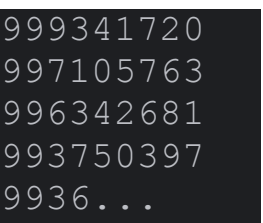
```

    tmp -= 1
    if tmp == 0:
        break
open("output.txt", "w").write(normVid(array))

```

- аналогично с заданием 1
- Заменяем > на <
- tests:

аналогично с предыдущими заданиями

ввод	Время	Память	Вывод
Пример из задачи	0.001000404357 9	15.19 МБ	
Минимальные данные	0.000979900360 10	15.27 МБ	
Максимальные данные	0.0209963	15.37 МБ	

-
-

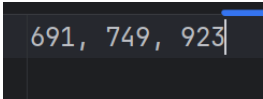
Задание 4

Напишите код линейного поиска, при работе которого выполняется сканирование последовательности в поисках значения V .

```
def lineSearch() -> None:
    file = open("input.txt").readlines()[0]
    elem = int(open("input.txt").readlines()[1])
    array = list(map(int, file.split()))
    indexes: list[int] = []

    for i in range(len(array)):
        if array[i] == elem:
            indexes.append(i)
    if len(indexes) == 0:
        open("output.txt", "w").write(str(-1))
        return
    if len(indexes) == 1:
        open("output.txt",
"w").write(str(indexes)[0])
        return
    open("output.txt", "w").write(str(indexes)[1:-1])
```

- Читаем файл, преобразуем в array, выполняем поиск
- записываем результат в output.txt
-

ввод	Время	Память	Вывод
Пример из задачи	-	-	-
Минимальные данные	0.000999	15.06 МБ	-1
Максимальные данные	0.0009999927520	15.46 МБ	

-

Задание 5

Рассмотрим сортировку элементов массива, которая выполняется следующим образом. Сначала определяется наименьший элемент массива, который ставится на место элемента $A[1]$. Затем производится поиск второго наименьшего элемента массива A , который ставится на место элемента $A[2]$. Этот процесс продолжается для первых $n - 1$ элементов массива A . Напишите код этого алгоритма

```
def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

def minimum(array: list[int]) -> int:
    if len(array) == 0:
        return -1

    array1 = [array[0], 0]

    for i in range(len(array)):
        if array[i] < array1[0]:
            array1[0], array1[1] = array[i], i
    return array1[1]
```

```

def maximum(array: list[int]) -> int:
    if len(array) == 0:
        return -1

    array1 = [array[0], 0]

    for i in range(len(array)):
        if array[i] > array1[0]:
            array1[0], array1[1] = array[i], i
    return array1[1]

def selectionSort() -> None:
    file = open("input.txt").readlines()[0]
    array = list(map(int, file.split()))
    sorted_array: list[int] = []

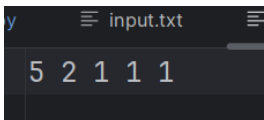
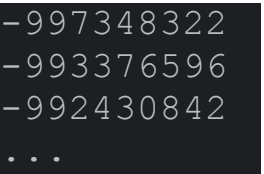
    for i in range(len(array)):

        sorted_array.append(array.pop(minimum(array)))
        open("output.txt",
            "w").write(str(normVid(sorted_array)))

```

- Создаем функцию поиска индекса минимального эл-та и максимального

- дальше в “отсортированный”(изначально пустой) массив добавляем минимальный элемент из неотсортированного и убираем его оттуда (.pop())
- В задаче требуется найти время сортировки в наихудшем случае, однако из-за того, что функция “minimum” выводит не наименьший эл-т, а его индекс, то время выполнения не зависит от массива и зависит лишь от числа эл-тов, а не их значений. НО если бы функция выводила бы минимальный элемент, то наихудшим был бы массив, отсортированный инверсивно, поскольку для нахождения минимального элемента пришлось бы проходить весь массив.

ввод	Время	Память	Вывод
Пример из задачи	0.0010004043579	15.19 МБ	
Минимальные данные	0.000997	15.11 МБ	0
Максимальные данные	0.013012	15.50 МБ	

Задание 6

Написать сортировку пузырьком

```
def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

def Bubble_sort():
    file = open("input.txt").readlines()[1]
    array = list(map(int, file.split()))

    for i in range(len(array)):
        for j in range(len(array)-1, i, -1):
            if array[j-1] > array[j]:

                tmp_1 = array[j-1]
                tmp_2 = array[j]

                array[j-1], array[j] = tmp_2, tmp_1
        print(array)
    open("output.txt", "w").write(normVid(array))
```

- переписан код из задания (изменен)

- ввод, вывод как в предыдущих заданиях

док-во корректности.

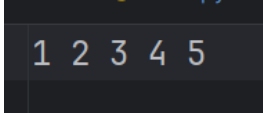
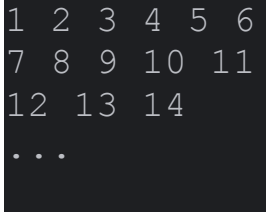
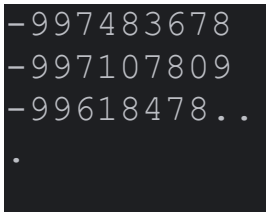
- 1) она завершается, т.к. количество итерация ограничено и равно $n*(n+1)/2$
- 2) хватит ли этих перестановок? да. допустим для каждого элемента расстояние от его нынешнего, до отсортированного индекса максимально, посчитаем:

для 1ого максимальное расстояние N, для второго n-1 и тд
сумма == $n*(n+1)/2$

док-во на практике для массива [5, 4, 3, 2, 1]:

```
[5, 4, 3, 1, 2]
[5, 4, 1, 3, 2]
[5, 1, 4, 3, 2]
[1, 5, 4, 3, 2]
[1, 5, 4, 2, 3]
[1, 5, 2, 4, 3]
[1, 2, 5, 4, 3]
[1, 2, 5, 3, 4]
[1, 2, 3, 5, 4]
[1, 2, 3, 4, 5]
```

этот пример показывает, что итераций хватает даже для
наихудшего случая (инверсивно отсортирован)

ввод	Время	Память	Вывод
Пример из задачи	0	15.46 МБ	
Минимальные данные	0	15.26 МБ	0
Максимальные данные (наихудший вариант)	0.0385980	15.40 МБ	
макс. средний вариант	0.02898073	15.74 МБ	

Вывод:

Сортировка пузырьком зависит не только от количества, но и от качества элементов в массиве. (время отличается)